

Streamlitapp.py

```
import pandas as pd
import time
import streamlit as st
import warnings
import os
import warnings
import sys
from io import StringIO
import contextlib
from tabulate import tabulate
import streamlit as st
import pytube
from pytube import YouTube
import transformers
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
from youtube_transcript_api import YouTubeTranscriptApi
from huggingface_hub import InferenceClient
import librosa
import soundfile as sf
from langchain.chains.llm import LLMChain
from langchain.prompts import PromptTemplate
from langchain.chains import ReduceDocumentsChain, MapReduceDocumentsChain
from langchain.chains.combine_documents.stuff import StuffDocumentsChain
from langchain import HuggingFaceHub
import gradio as gr
from PIL import Image
from classes import get_primer, format_question, run_request,
get_text_primer

warnings.filterwarnings("ignore")
st.set_option('deprecation.showPyplotGlobalUse', False)
st.set_page_config(layout="wide", page_title="DocSense")

# image = Image.open('.streamlit/docsense logo.png')
# new_image = image.resize((475, 128))
# st.image(new_image)

st.markdown(
    "<img src='https://i.ibb.co/Qjkv5Tv/docsense-logo.png' style='height: 120px; width: 475px; display: block; margin: 0 auto;*>",
    unsafe_allow_html=True)

st.markdown(
    "<h4 style='text-align: center;padding-top: 0rem;*>by Adarsh Agarwal, Saakar Chaudhary & Farhan Shaikh</h2*>",
    unsafe_allow_html=True)

hf_key = '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'

placeholder = st.empty()

app_selection = st.sidebar.radio("Select the app", ["Derive insights from CSV", "Summarize YouTube Video"])

if app_selection == "Derive insights from CSV":
    placeholder.empty()
```

```

if "datasets" not in st.session_state:
    datasets = {}
    # Preload datasets
    datasets["Movies"] = pd.read_csv("movies.csv")
    datasets["Cars"] = pd.read_csv("cars.csv")
    datasets["Colleges"] = pd.read_csv("colleges.csv")
    st.session_state["datasets"] = datasets
else:

    datasets = st.session_state["datasets"]

with placeholder.container():
    with st.sidebar:
        mode = st.radio("Select Mode:", ("Visualize", "Chat"))

        dataset_container = st.empty()

        try:
            uploaded_file = st.file_uploader(":computer: Load a CSV
file:", type="csv")
            index_no = 0
            if uploaded_file:
                file_name = uploaded_file.name[:-4].capitalize()
                datasets[file_name] = pd.read_csv(uploaded_file)

                index_no = len(datasets) - 1
            except Exception as e:
                st.error("File failed to load. Please select a valid CSV
file.")

                print("File failed to load.\n" + str(e))

            chosen_dataset = dataset_container.radio(":bar_chart: Choose
your data:", datasets.keys(),
                                                    index=index_no) #
,horizontal=True,)

            if mode == "Visualize":
                question = st.text_area(":eyes: What would you like to
visualise?", height=10)
                go_btn = st.button("Go...")

                if go_btn:
                    if not hf_key.startswith('hf_'):
                        st.error("Please enter a valid HuggingFace API key.")
                    else:
                        st.subheader("Code Llama")
                        try:
                            primer1, primer2 =
get_primer(datasets[chosen_dataset], 'datasets["' + chosen_dataset + "']')

                            question_to_ask = format_question(primer1, primer2,
question, "Code Llama")

                            answer = run_request(question_to_ask, "CodeLlama-
34b-Instruct-hf", alt_key=hf_key)

                            answer = primer2 + answer
                            plot_area = st.empty()
                            plot_area.pyplot(exec(answer))
                            st.code(answer)

```

```

        except Exception as e:
            st.error(f"An error occurred: {e}")

    elif mode == "Chat":
        chat_question = st.text_area("What question do you have about
the dataset?", height=10)
        chat_btn = st.button("Ask")

        @contextlib.contextmanager
        def stdoutIO(stdout=None):
            old = sys.stdout
            if stdout is None:
                stdout = StringIO()
            sys.stdout = stdout
            yield stdout
            sys.stdout = old

        if chat_btn:
            try:
                output_dict = {}
                primer1, primer2 =
get_text_primer(datasets[chosen_dataset], 'datasets["' + chosen_dataset +
"']')
                question_to_ask = format_question(primer1, primer2,
chat_question, "Code Llama")
                answer = run_request(question_to_ask, "CodeLlama-34b-
Instruct-hf", alt_key=hf_key)
                answer = primer2 + answer
                output_dict['datasets'] = datasets
                op_area = st.empty()
                with op_area.container():
                    with stdoutIO() as s:
                        exec(answer)
                        st.markdown(s.getvalue(), unsafe_allow_html=True)
                        st.code(answer)
            except Exception as e:
                st.error(f"An error occurred: {e}")

    tab_list = st.tabs(datasets.keys())

    for dataset_num, tab in enumerate(tab_list):
        with tab:
            dataset_name = list(datasets.keys())[dataset_num]
            st.subheader(dataset_name)
            st.dataframe(datasets[dataset_name], hide_index=True)

    footer = """<style>.footer {position: fixed;left: 0;bottom:
0;width: 100%;text-align: center;}</style><div class="footer">
    <p> <a style='display: block; text-align: center;'>
Datasets courtesy of NL4DV, nvBench and ADVISor </a></p></div>"""
    st.caption("Datasets courtesy of NL4DV, nvBench and ADVISor")

    hide_streamlit_style = """
        <style>
            #MainMenu {visibility: hidden;}
            footer {visibility: hidden;}
        </style>
        """

```

```

        st.markdown(hide_streamlit_style, unsafe_allow_html=True)

elif app_selection == "Summarize YouTube Video":
    placeholder.empty()
    transcription_model_id = "openai/whisper-large"
    llm_model_id = "tiiuae/falcon-7b-instruct"
    HF_TOKEN = os.environ.get("HF_TOKEN", None)
    save_dir = "docs"

    def get_youtube_title(url):
        yt = YouTube(url)
        return yt.title

    def summarize_youtube_video(url, force_transcribe, hf_key, temperature,
words, do_sample):
        save_dir = os.path.join('./', 'docs')
        if not os.path.exists(save_dir):
            os.mkdir(save_dir)

        title, text, transcript_source = transcribe_youtube_video(url,
force_transcribe, True, hf_key)

        summary, summary_source = summarize_text(title, text, temperature,
words, True, hf_key, do_sample)

        return title, text, transcript_source, summary, summary_source

    def transcribe_youtube_video(url, force_transcribe, use_api, hf_key):
        yt = YouTube(str(url))
        text = ''

        try:
            text = get_yt_transcript(url)
        except:
            pass

        if text == '' or force_transcribe:
            if use_api:
                text = transcribe_yt_vid_api(url, hf_key=hf_key)
                transcript_source = 'The transcript was generated using {}
via the Hugging Face Hub API.'.format(
                    transcription_model_id)
            else:
                text = transcribe_yt_vid(url)
                transcript_source = 'The transcript was generated using {}
hosted locally.'.format(
                    transcription_model_id)
            else:
                transcript_source = 'The transcript was downloaded from
YouTube.'

        return yt.title, text, transcript_source

    def get_yt_transcript(url):
        text = ''
        vid_id = pytube.extract.video_id(url)
        temp = YouTubeTranscriptApi.get_transcript(vid_id)

```

```

        for t in temp:
            text += t['text'] + ' '
        return text

def transcribe_yt_vid_api(url, hf_key):
    yt = YouTube(str(url))
    audio = yt.streams.filter(only_audio=True).first()
    out_file = audio.download(filename="audio.wav",
output_path=save_dir)

    text = ''
    t = 25
    x, sr = librosa.load(out_file, sr=None)

    for i in range(0, (len(x) // (t * sr)) + 1):
        y = x[t * sr * i: t * sr * (i + 1)]
        split_path = os.path.join(save_dir, "audio_split.wav")
        sf.write(split_path, y, sr)
        text += client.automatic_speech_recognition(split_path)

    return text

def summarize_text(title, text, temperature, words, use_api, hf_key,
do_sample):

    from langchain.chains.llm import LLMChain
    from langchain.prompts import PromptTemplate
    from langchain.chains import ReduceDocumentsChain,
MapReduceDocumentsChain
    from langchain.chains.combine_documents.stuff import
StuffDocumentsChain
    import torch
    import transformers
    from transformers import BitsAndBytesConfig
    from transformers import AutoTokenizer, AutoModelForCausalLM

    from langchain import HuggingFacePipeline
    import torch

    model_kwargs1 = {"temperature": temperature,
                    "do_sample": do_sample,
                    "min_new_tokens": 200 - 25,
                    "max_new_tokens": 200 + 25
                    }
    model_kwargs2 = {"temperature": temperature,
                    "do_sample": do_sample,
                    "min_new_tokens": words - 25,
                    "max_new_tokens": words + 25,
                    'repetition_penalty': 2.0
                    }

    if not do_sample:
        del model_kwargs1["temperature"]
        del model_kwargs2["temperature"]

    if use_api:

        llm = HuggingFaceHub(
            repo_id=llm_model_id, model_kwargs=model_kwargs1,
            huggingfacehub_api_token=hf_key

```

```

    )
    llm2 = HuggingFaceHub(
        repo_id=llm_model_id, model_kwargs=model_kwargs2,
        huggingfacehub_api_token=hf_key
    )
    summary_source = 'The summary was generated using {} via
Hugging Face API.'.format(llm_model_id)

else:
    quantization_config = BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_compute_dtype=torch.float16,
        bnb_4bit_quant_type="nf4",
        bnb_4bit_use_double_quant=True,
    )

    tokenizer = AutoTokenizer.from_pretrained(llm_model_id)
    model = AutoModelForCausalLM.from_pretrained(llm_model_id,

                                                )

    model.to_bettertransformer()

    pipeline = transformers.pipeline(
        "text-generation",
        model=model,
        tokenizer=tokenizer,
        torch_dtype=torch.bfloat16,
        device_map="auto",
        pad_token_id=tokenizer.eos_token_id,
        **model_kwargs1,
    )
    pipeline2 = transformers.pipeline(
        "text-generation",
        model=model,
        tokenizer=tokenizer,
        torch_dtype=torch.bfloat16,
        device_map="auto",
        pad_token_id=tokenizer.eos_token_id,
        **model_kwargs2,
    )
    llm = HuggingFacePipeline(pipeline=pipeline)
    llm2 = HuggingFacePipeline(pipeline=pipeline2)

    summary_source = 'The summary was generated using {} hosted
locally.'.format(llm_model_id)

    map_template = """
        You are an intelligent AI assistant that is tasked to
review the content of a video and provide a concise and accurate summary.\n
        You do not provide information that is not mentioned in the
video. You only provide information that you are absolutely sure about.\n
        Summarize the following text in a clear and concise way:\n
        ----- \n
        TITLE: `{title}`\n
        TEXT:\n
        `{docs}`\n
        ----- \n
        BRIEF SUMMARY:\n
        """

    map_prompt = PromptTemplate(
        template=map_template,

```

```

        input_variables=['title', 'docs']
    )
    map_chain = LLMChain(llm=llm, prompt=map_prompt)

    collapse_template = """
        You are an intelligent AI assistant that is tasked to
review the content of a video and provide a concise and accurate summary.\n
        You do not provide information that is not mentioned in the
video. You only provide information that you are absolutely sure about.\n
        The following is set of partial summaries of a video:\n
        ----- \n
        TITLE: `{title}`\n
        PARTIAL SUMMARIES:\n
        `{doc_summaries}`\n
        ----- \n
        Take these and distill them into a consolidated summary.\n
        SUMMARY:\n
        """

    collapse_prompt = PromptTemplate(
        template=collapse_template,
        input_variables=['title', 'doc_summaries']
    )
    collapse_chain = LLMChain(llm=llm, prompt=collapse_prompt)

    collapse_documents_chain = StuffDocumentsChain(
        llm_chain=collapse_chain,
document_variable_name="doc_summaries"
    )

    combine_template = """\n
        You are an intelligent AI assistant that is tasked to
review the content of a video and provide a concise and accurate summary.\n
        You do not provide information that is not mentioned in the
video. You only provide information that you are absolutely sure about.\n
        The following is a set of partial summaries of a video:\n
        ----- \n
        TITLE: `{title}`\n
        PARTIAL SUMMARIES:\n
        `{doc_summaries}`\n
        ----- \n
        Generate an executive summary of the whole text in maximum
{words} words that contains the main messages, points, and arguments
presented in the video as bullet points.\n
        EXECUTIVE SUMMARY:\n
        """

    combine_prompt = PromptTemplate(
        template=combine_template,
        input_variables=['title', 'doc_summaries', 'words']
    )
    combine_chain = LLMChain(llm=llm2, prompt=combine_prompt)

    combine_documents_chain = StuffDocumentsChain(
        llm_chain=combine_chain, document_variable_name="doc_summaries"
    )

    reduce_documents_chain = ReduceDocumentsChain(

        combine_documents_chain=combine_documents_chain,

        collapse_documents_chain=collapse_documents_chain,

```

```

        token_max=800,
    )

    map_reduce_chain = MapReduceDocumentsChain(

        llm_chain=map_chain,

        reduce_documents_chain=reduce_documents_chain,

        document_variable_name="docs",

        return_intermediate_steps=False,
    )

    from langchain.document_loaders import TextLoader
    from langchain.text_splitter import TokenTextSplitter

    if not os.path.exists(save_dir):
        os.makedirs(save_dir)

    transcript_file_path = os.path.join(save_dir, 'transcript.txt')
    with open(save_dir + '/transcript.txt', 'w') as f:
        f.write(text)
    loader = TextLoader(save_dir + "/transcript.txt")
    doc = loader.load()
    text_splitter = TokenTextSplitter(chunk_size=800,
chunk_overlap=100)
    docs = text_splitter.split_documents(doc)

    summary = map_reduce_chain.run({'input_documents': docs, 'title':
title, 'words': words})

    try:
        del (map_reduce_chain, reduce_documents_chain, combine_chain,
collapse_documents_chain, map_chain,
            collapse_chain, llm, llm2, pipeline, pipeline2, model,
tokenizer)
    except:
        pass
    torch.cuda.empty_cache()

    return summary, summary_source

import gradio as gr
import pytube
from pytube import YouTube

force_transcribe = False
temperature = st.sidebar.slider("Generation temperature",
min_value=0.01, max_value=1.0, value=0.25)
words = st.sidebar.slider("Length of the summary", min_value=100,
max_value=500, value=100)
do_sample = st.sidebar.checkbox("Set the Temperature")

with placeholder.container():

    url = st.text_input("Enter YouTube video URL:", "")
    if st.button("Summarize!"):
        if not url:

```

```

        st.error("Please enter a valid YouTube URL.")
    else:
        with st.spinner('Processing, please wait....'):
            time.sleep(7)
            st.success('Done!')
            title, text, transcript_source =
transcribe_youtube_video(url, force_transcribe, True, hf_key)
            summary, summary_source = summarize_text(title, text,
temperature, words, True, hf_key, do_sample)

            st.subheader(title, divider='rainbow')
            st.caption(transcript_source)
            st.text(summary)

```

Classes.py

```

from langchain import HuggingFaceHub, LLMChain, PromptTemplate

def run_request(question_to_ask, model_type, alt_key):
    # Hugging Face model
    llm = HuggingFaceHub(huggingfacehub_api_token=alt_key,
repo_id="codellama/" + model_type,
                        model_kwargs={"temperature": 0.1,
"max_new_tokens": 500})
    llm_prompt = PromptTemplate.from_template(question_to_ask)
    llm_chain = LLMChain(llm=llm, prompt=llm_prompt)
    llm_response = llm_chain.predict()
    # rejig the response
    llm_response = format_response(llm_response)
    return llm_response

def format_response(res):
    # Remove the load_csv from the answer if it exists
    csv_line = res.find("read_csv")
    if csv_line > 0:
        return_before_csv_line = res[0:csv_line].rfind("\n")
        if return_before_csv_line == -1:
            # The read_csv line is the first line so there is nothing to
need before it
            res_before = ""
        else:
            res_before = res[0:return_before_csv_line]
        res_after = res[csv_line:]
        return_after_csv_line = res_after.find("\n")
        if return_after_csv_line == -1:
            # The read_csv is the last line
            res_after = ""
        else:
            res_after = res_after[return_after_csv_line:]
        res = res_before + res_after
    return res

```

```

def format_question(primer_desc, primer_code, question, model_type):
    # Fill in the model_specific_instructions variable
    instructions = "\nDo not use the 'c' argument in the plot function, use
'color' instead and only pass color names like 'green', 'red', 'blue'."
    primer_desc = primer_desc.format(instructions)
    # Put the question at the end of the description primer within quotes,
then add on the code primer.
    return ""'\n' + primer_desc + question + '\n""'\n' + primer_code

def get_primer(df_dataset, df_name):
    # Primer function to take a dataframe and its name
    # and the name of the columns
    # and any columns with less than 20 unique values it adds the values to
the primer
    # and horizontal grid lines and labeling
    primer_desc = "Use a dataframe called df from data_file.csv with
columns '"\
        + "','".join(str(x) for x in df_dataset.columns) + "'. "
    for i in df_dataset.columns:
        if len(df_dataset[i].drop_duplicates()) < 20 and
df_dataset.dtypes[i] == "O":
            primer_desc = primer_desc + "\nThe column '" + i + "' has
categorical values '" + \
                "','".join(str(x) for x in
df_dataset[i].drop_duplicates()) + "'. "
            elif df_dataset.dtypes[i] == "int64" or df_dataset.dtypes[i] ==
"float64":
                primer_desc = primer_desc + "\nThe column '" + i + "' is type "
+ str(
                    df_dataset.dtypes[i]) + " and contains numeric values. "
            primer_desc = primer_desc + "\nLabel the x and y axes appropriately."
            primer_desc = primer_desc + "\nAdd a title. Don't put any fig
suptitle."
            primer_desc = primer_desc + "{}" # Space for additional instructions
if needed
            primer_desc = primer_desc + "\nUsing Python version 3.9.12, create a
script using the dataframe df to graph the following: "
            pimer_code = "import pandas as pd\nimport matplotlib.pyplot as plt\n"
            pimer_code = pimer_code + "fig,ax = plt.subplots(1,1,figsize=(10,4))\n"
            pimer_code = pimer_code +
"ax.spines['top'].set_visible(False)\nax.spines['right'].set_visible(False)
\n"
            pimer_code = pimer_code + "df=" + df_name + ".copy()\n"
            return primer_desc, pimer_code

def get_text_primer(df_dataset, df_name):
    # Primer function to take a dataframe
    # and the name of the columns
    # and any columns with less than 20 unique values it adds the values to
the primer
    primer_desc = "Use a dataframe called df from data_file.csv with
columns '"\
        + "','".join(str(x) for x in df_dataset.columns) + "'. "
    for i in df_dataset.columns:
        if len(df_dataset[i].drop_duplicates()) < 20 and
df_dataset.dtypes[i] == "O":
            primer_desc = primer_desc + "\nThe column '" + i + "' has

```

```

categorical values '" + \
    "','".join(str(x) for x in
df_dataset[i].drop_duplicates()) + "'. "
    elif df_dataset.dtypes[i] == "int64" or df_dataset.dtypes[i] ==
"float64":
        primer_desc = primer_desc + "\nThe column '" + i + "' is of
type " + str(
            df_dataset.dtypes[i]) + " and contains numeric values. "
        primer_desc = primer_desc + "\nPlease ensure that the output is
generated using 'print' statement with tabulate only, keep tablefmt as html
and showindex=False.\n Make sure to show headers of the dataframe."
        primer_desc = primer_desc + "\nUsing Python version 3.9.12, create a
script using the dataframe df to answer the following:"
        primer_code = "import pandas as pd \nfrom tabulate import tabulate \n"
        primer_code = primer_code + "df=" + df_name + ".copy()\n"
    return primer_desc, primer_code

```